Subject: ASD&D; Semester: V
Class: TE Batch: T11
Name: Arya Bailur Roll No: 08

Experiment 6

Aim: To explore and implement an Agile Software Project lifecycle using Jira integrated with a DevOps Pipeline.

Lab Outcome: Mapped LO1

LO1: Apply Agile methodologies and leverage Jira for Sprint Planning, task tracking, and integration with DevOps workflows.

Theory:

What is Jira?

Jira is an Agile project management and issue-tracking platform built by Atlassian. It enables software teams to plan sprints, manage tasks, track defects, and oversee releases through highly customizable workflows.

Core Concepts & Hierarchy

- **Project:** A collaborative workspace that represents a product, application, or team initiative.
- **Issue Hierarchy:**
  - **Epic:** A broad objective or high-level feature decomposed into smaller units of work (e.g., Core Infrastructure).
  - **Story:** An end-user feature expressed from the user's perspective (e.g., Storefront UI).
  - **Task / Bug:** Technical work items or defects that need resolution.
  - **Sub-task:** Fine-grained steps within a story or task.
- **Product Backlog:** A prioritized queue of all outstanding features, improvements, and bug fixes awaiting assignment.
- **Sprint & Scrum Board:** A time-boxed iteration (usually 1–2 weeks) during which team members pull items from the backlog into workflow columns (To Do, In Progress, In Review, Done).
- **Story Points:** Relative estimates that gauge effort, technical complexity, and risk for each work item.

Jira Setup & Deployment Models

- **Jira Cloud (SaaS):** Hosted by Atlassian with no server maintenance required. Teams create an organization site (e.g., company.atlassian.net) and gain immediate browser-based access.
- **Jira Data Center (Self-Managed):** Deployed on private infrastructure or dedicated cloud servers (AWS/Azure) for organizations that need localized data governance, custom security policies, and high-availability clustering.

DevOps Integration & Automation Lifecycle

• **Version Control Linkage:** Connects with code repositories like GitHub, GitLab, or Bitbucket via official marketplace apps (e.g., GitHub for Jira).

• **Smart Issue Keys:** Including ticket identifiers (e.g., SCRUM-4, SCRUM-6) in branch names, commit messages, or Pull Request titles automatically links VCS activity to the corresponding Jira issue panel.

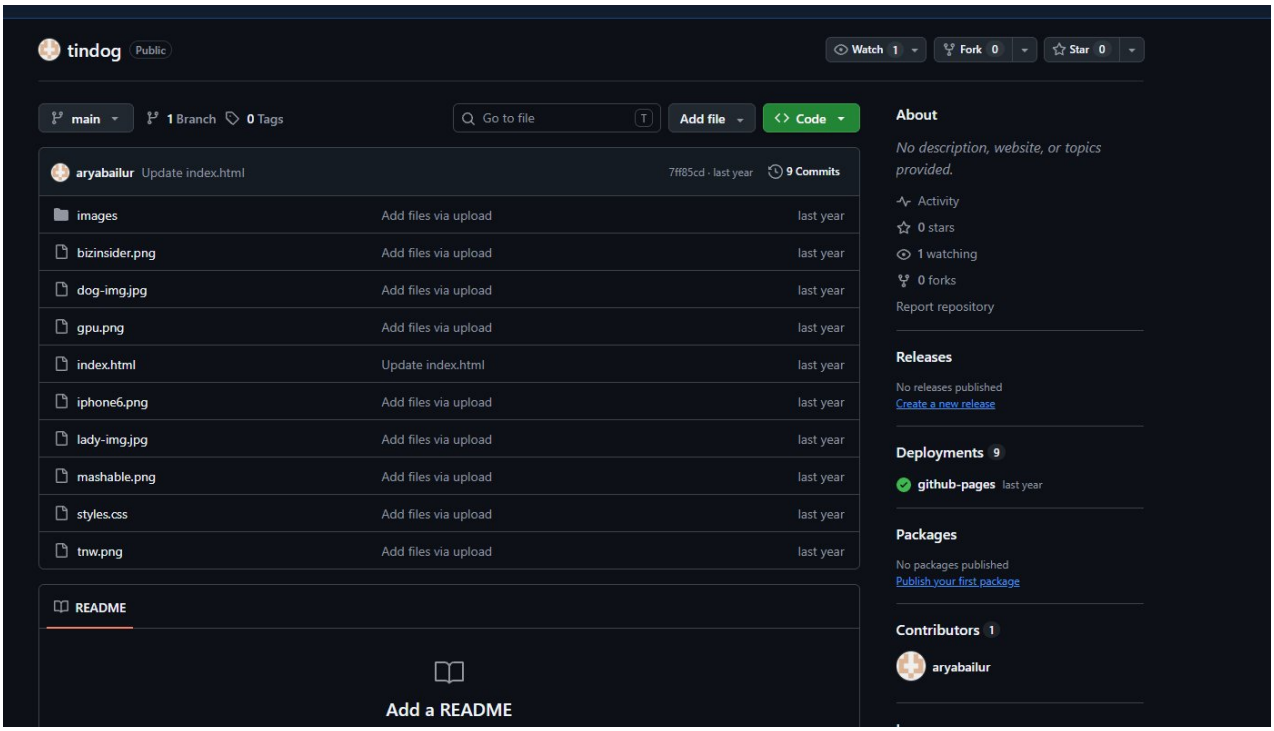
• **Jira Automation Rules:** Event-driven rules that trigger state transitions on the board (e.g., moving a ticket to In Review when a Pull Request is opened, or transitioning to Done when a PR is merged).

Team Collaboration & Access Management

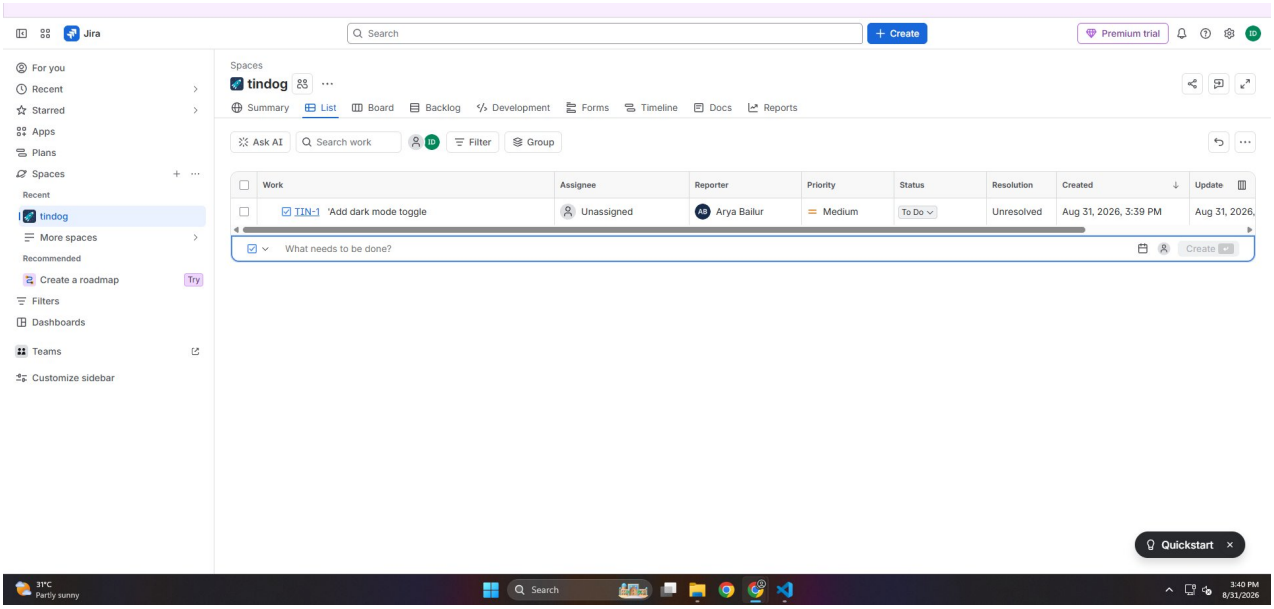
• **Role-Based Access Control (RBAC):** Access is governed at both the organization and project levels through roles such as Administrator, Member, and Viewer.

• **Work Allocation:** Tickets are assigned to individual developers, providing visibility into sprint capacity, bottlenecks, and velocity reports.

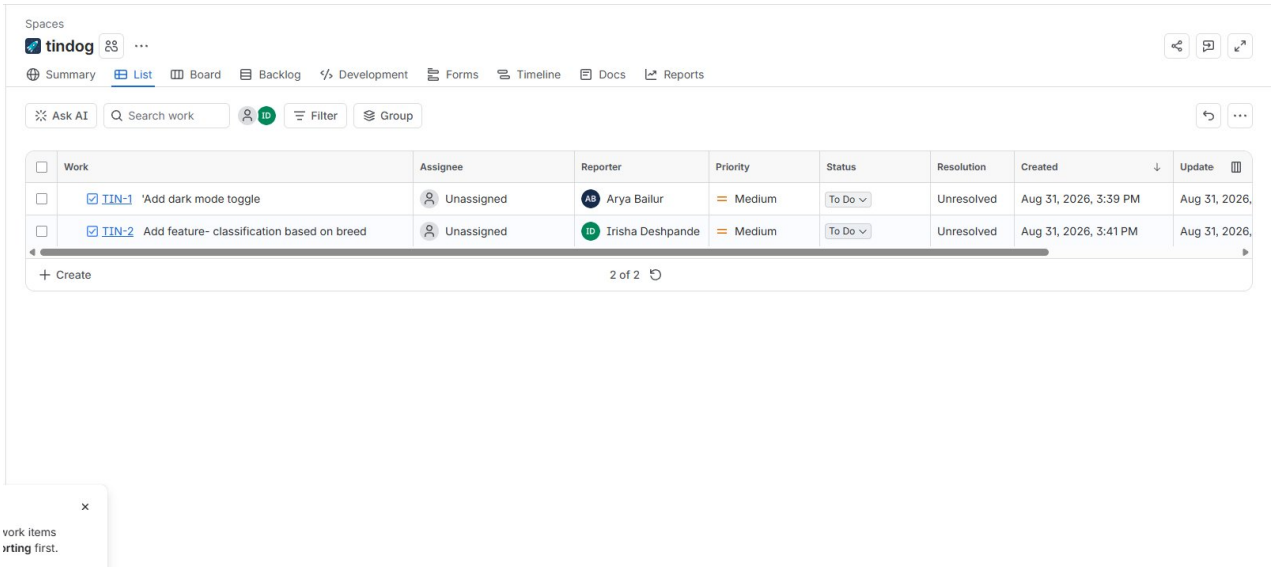
Implementation:



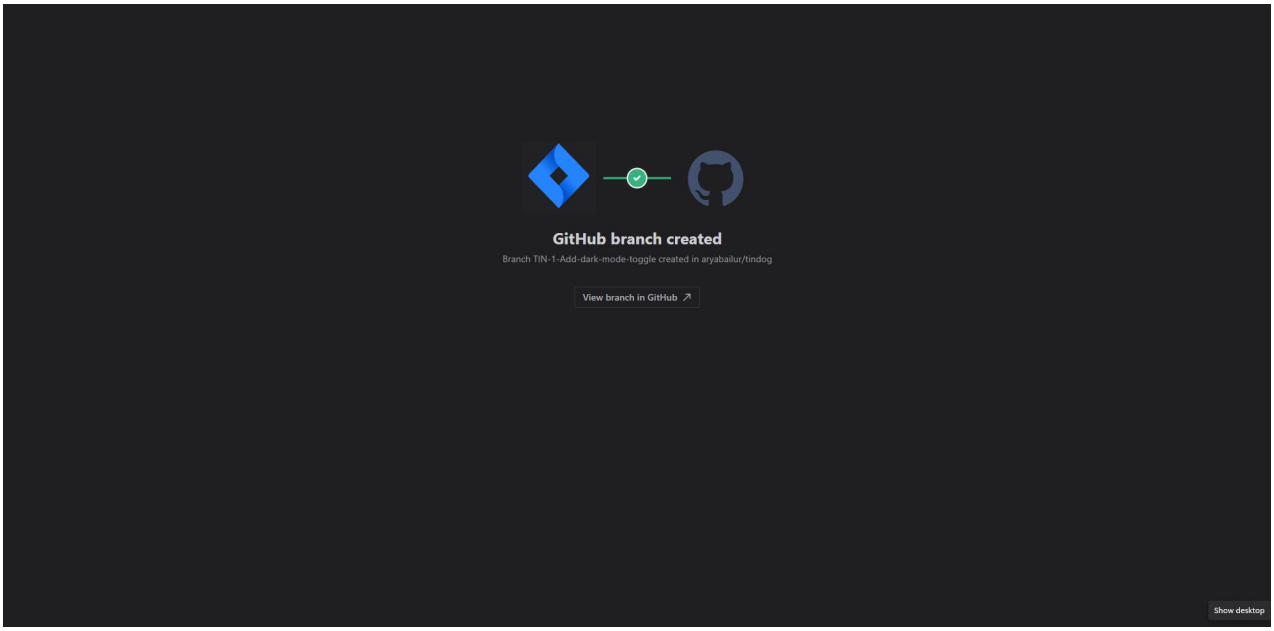
GitHub repository overview



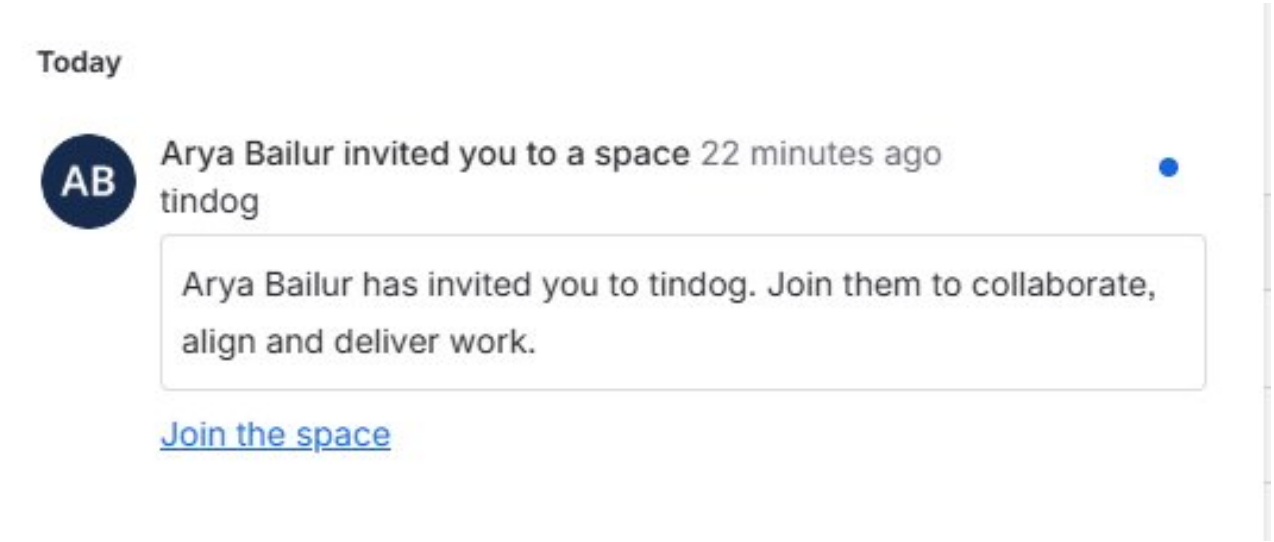
Jira project board with backlog items



Jira list view – task assignments and status



GitHub branch created and linked to Jira issue



Team invitation to the Jira space

Conclusion:

Integrating Jira with GitHub established a streamlined, end-to-end Agile development lifecycle for our E-commerce application. Through this experiment, we learned how to organize backlogs into active sprints, distribute tasks among team members, and track real-time progress via Scrum boards. By referencing issue keys in branch names and Pull Requests, we achieved seamless traceability between the codebase and Jira tickets. Additionally, configuring GitHub Actions CI pipelines alongside Jira Automation illustrated how contemporary DevOps workflows reduce manual overhead by automating testing and board transitions upon code merges.